A User-Centred, From-Scratch Scientific Calculator for the Tangent Function $\tan(x)$

Deliverable 2: From-Scratch Implementation, Tkinter GUI, VCS, Updated Requirements

Risat Ahmed
Student ID: 40294116

SOEN 6011 – Software Engineering Processes, Section CC
Concordia University – Summer 2026

July 29, 2026

Outline

1. From-Scratch Boundary
2. Numerical Core
3. Tkinter GUI
4. Repository & README
5. Updated Requirements
6. Verification
7. GAI Prompts
8. References

Problem 5 — Interpreting "From Scratch"

D1 status: sin/cos already from scratch (Maclaurin series); only `PI = math.pi` remained.

Interpretation adopted (decision D-06):

- **Prohibited:** a library call that itself computes a trig/transcendental result (`math.sin/cos/tan/atan/pi`).
- **Permitted:** ordinary arithmetic (`+ - * /`), I/O, the Tkinter toolkit.

Consequence: Algorithm A (Maclaurin) already satisfies the constraint once `PI` is independent $\Rightarrow$ **retained**, reversing D1's forward note toward CORDIC.

Verified: `grep -rn "math\." src/` $\rightarrow$ no matches outside prose.

From-Scratch π — Machin's Identity

$$\frac{\pi}{4} = 4\arctan\left(\tfrac{1}{5}\right) - \arctan\left(\tfrac{1}{239}\right)$$

```
def _arctan_series(y, tol, n_max):
    term = y; total = y; y2 = y*y; n = 1
    while abs(term) >= tol and n <= n_max:
        term = term * (-y2) * (2*n-1) / (2*n+1)
        total += term; n += 1
    return total

PI = 16*_arctan_series(1/5) - 4*_arctan_series(1/239)
```

Converges in $\sim$13 terms to 10^{-18}; matches `math.pi` to 8.9×10^{-16}.

Numerical Architecture (D2-P5.2)

`constants.py` – from-scratch `PI`
`exceptions.py` – `NumericalRangeError`, `DomainError`, `ConvergenceError`
`trig_series.py` – `maclaurin_sin/cos`

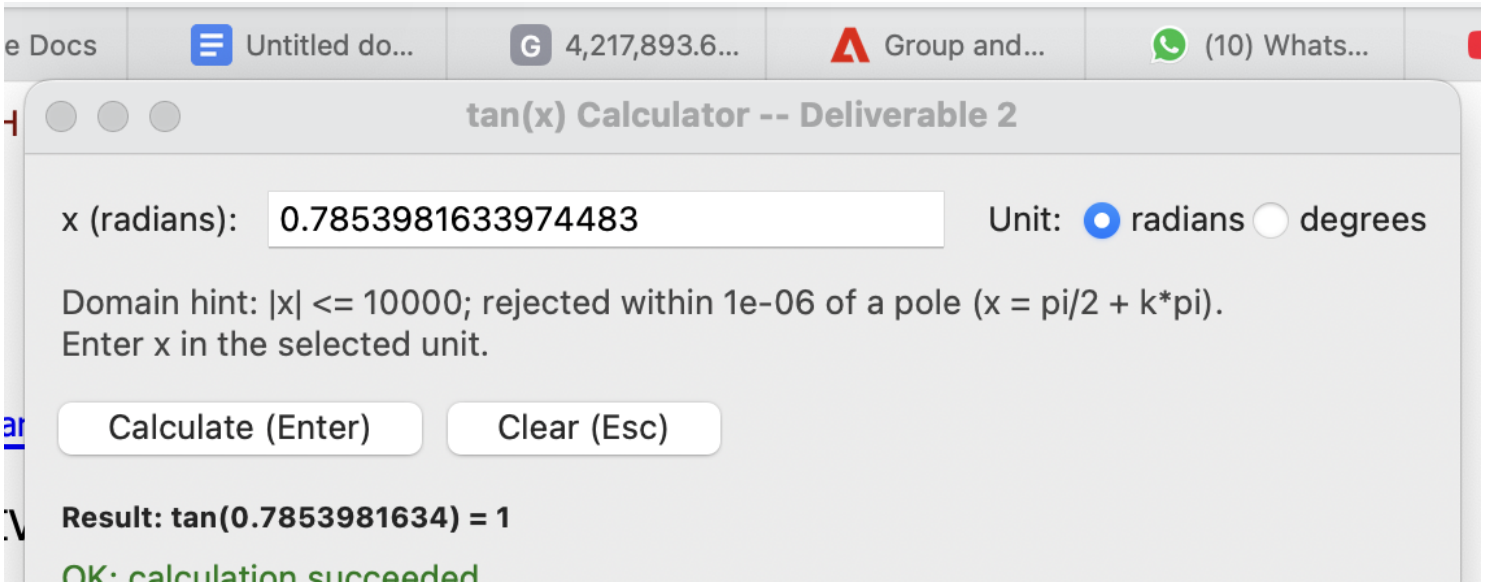
`validation.py` – parsing + deg/rad
`tan_core.py` – pure `tan(x, tolerance, max_terms)`, no I/O

Series functions now **raise** `ConvergenceError` on exhaustion instead of D1's silent stop – closes a REL-01 gap. Core is importable/testable without launching the GUI; both `cli.py` and `gui.py` call it as their only numerical dependency.

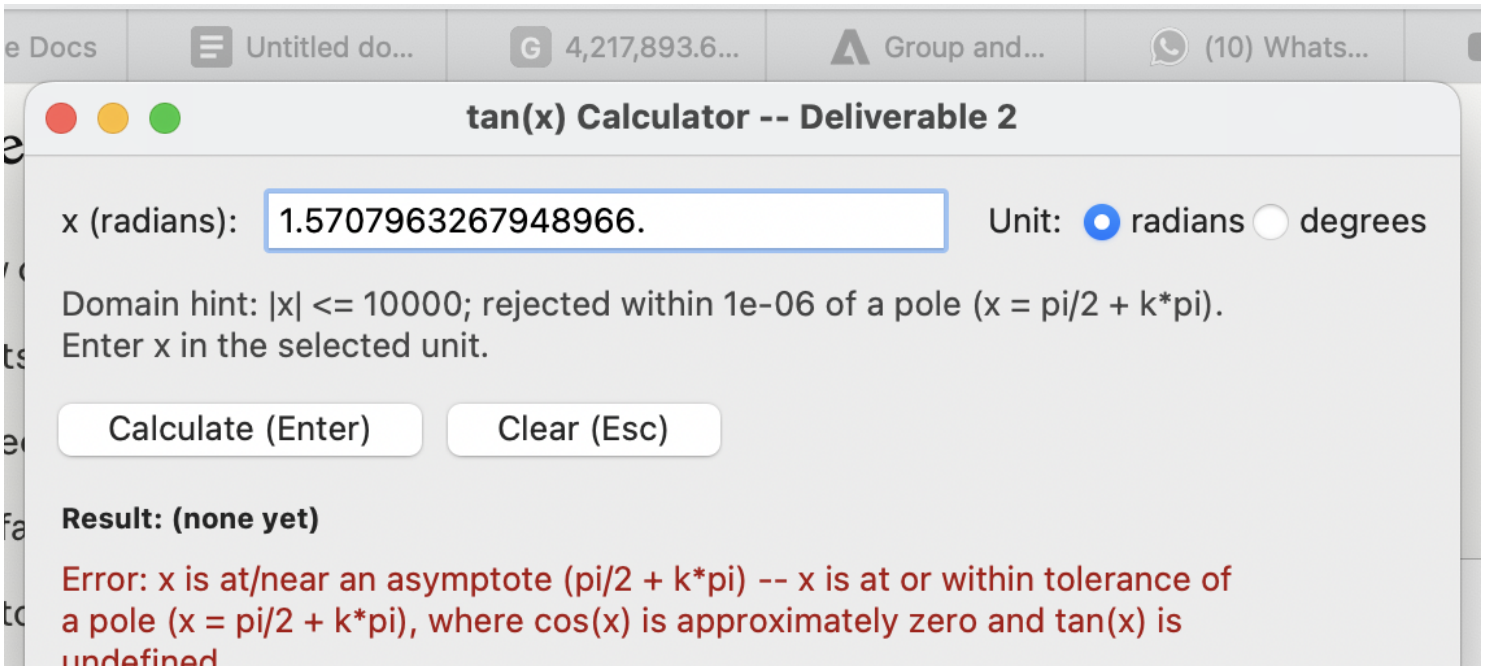
Problem 5 — GUI Wireframe (before code)

- x entry, label updates with unit ("x in radians"/"x in degrees")
- Radians/degrees radio buttons, radians default
- Domain hint shown *before* any error can occur
- Calculate/Clear buttons + Enter/Esc key bindings
- Status text always prefixed "Error"/"OK" – never colour alone

GUI — Valid Result



GUI — Near-Pole Rejection (No Traceback)



Problem 6 — Repository & Commit History

Public repo: `https://github.com/risatahmed/soen6011-tan-calculator`

- D1 submitted as a Moodle zip before Git init – stated honestly as one baseline commit, not fabricated/backdated history
- D2: **11 cohesive, per-concern commits** (boundary doc → core → CLI migration → wireframe → GUI → verification → README → requirements → decision/GAI logs → report)
- `README.md`: purpose, run commands (`python3 -m src.cli`/`src.gui`), usage examples, error/troubleshooting table, honest known-limitations section

Problem 7 — Baseline v0.2 Changelog

ID	Change	What changed
FR-07	New	Labelled degrees input mode (D-08)
USE-03	New	Tkinter GUI, keyboard nav, no colour-only errors
DEP-01	New	Core shall not call `math.sin/cos/tan/pi`
ERR-03	New	3 named exception types, not bare `Exception`
ACC-01	Revised	Bound unchanged; near-pole margin (3.83×10^{-10}) flagged tight, not weakened

13 v0.1 requirements unchanged and still in force. v0.1 retained on disk (C-06).

Verification — 15/15 Cases Pass

Case	Result	Verdict
$x = \pi/4$	rel. err. 1.4×10^{-15}	Pass
$x = 1000$	rel. err. 5.6×10^{-13}	Pass
$x = \pi/2 - 10^{-6}$ (near pole)	rel. err. 3.8×10^{-10}	Pass (tightest)
$x = \pi/2$ (at pole)	`DomainError` raised	Pass (rejected)
$x = 10^{10}$	`NumericalRangeError` raised	Pass (rejected)

Reference: independent `math.tan`, never the algorithm verifying itself. Full matrix: `docs/verification_matrix_d2.md`.

Known Limitations (Honest Disclosure)

- $|x| \leq 10^4$ range bound unchanged, still provisional pending professor confirmation
- Near-pole relative error (3.8×10^{-10}) is the tightest margin to the 10^{-9} ceiling of any passing case
- No performance/timing requirement verified (deliberately out of scope)
- Formal PyUnit suite is a D3 deliverable; D2 verification is a standalone script

GAI Prompt — Problem 5 (Core + GUI)

Tool: Claude Code (Sonnet 5) **Type:** Generative + architectural

TASK Freeze the from-scratch boundary; design/implement a pure `tan(x, tolerance, max_terms)` core with named exceptions; wireframe then implement a Tkinter GUI; verify against references.

RESTRICTIONS No `math.sin/cos/tan/pi` in the compute path; core importable without the GUI; no unhandled traceback.

OUTPUT FORMAT Boundary doc, core modules, wireframe doc, `gui.py`, verification matrix.

AUDIENCE SOEN 6011 grading audience – evidence-based, traceable to VAL/ACC/ERR/REL requirements.

GAI Prompt — Problem 6 & 7

Problem 6 — **TASK:** cohesive incremental commits by concern; a README a stranger can run the project from. **RESTRICTIONS:** no secrets/IDE files committed; README claims checked, not assumed.

Problem 7 — **TASK:** compare every v0.1 requirement against D2 evidence; add/revise with a "discovered via" note; freeze v0.2. **RESTRICTIONS:** keep v0.1 on disk; no silent deletions; no unverifiable requirement wording (e.g. "visually appealing" rejected, replaced with concrete USE-03).

References

- ISO/IEC/IEEE 29148:2018, Requirements Engineering.
- J. E. Volder, "The CORDIC Trigonometric Computing Technique," *IRE Trans. Electronic Computers*, 1959.
- J.-M. Muller et al., *Handbook of Floating-Point Arithmetic*, 2nd ed., 2016.
- NIST Digital Library of Mathematical Functions, `https://dlmf.nist.gov/`.

Thank You

Questions?